

Demo en vivo — Historia de usuario: Cliente histórico (María)

Caso 1 · Almacenes Paris, Marketplace de Mejoramiento del Hogar · DSY1103 EP3 · Microservicios Spring Boot en Render

«María ingresa con su correo habitual; el sistema valida sus credenciales contra la API legacy. Busca un taladro, elige entre ofertas de distintos vendedores, paga y sigue el estado del despacho desde su perfil. Cuando el producto llega defectuoso, abre un reclamo; el Administrador media la disputa y autoriza el reembolso.»

La historia recorre **8 microservicios** (legacy, clientes, proveedores, productos, ventas, pagos, despacho, tickets) más notificaciones y administrador de apoyo, todos desplegados en `https://paris-<servicio>.onrender.com` con base de datos independiente en Neon, comunicación WebClient con validaciones cruzadas y seguridad JWT por rol.

Opción A — Demo automática (recomendada)

El script ejecuta TODO (pre-calentado incluido) pausando entre actos con Enter para narrar, mostrando cada petición y su respuesta:

```
cd Casol
./scripts/demo-maria.sh          # contra Render (nube)
./scripts/demo-maria.sh local    # contra localhost (antes: ./scripts/start-all.sh)
```

- Cada corrida usa un **cliente legacy nuevo** (contador automático) para mostrar la migración en vivo.
- Verifica y repara el escenario: repone stock si falta y renueva la oferta si expiró.
- Requiere `curl` y `jq`.

Opción B — Peticiones manuales, acto por acto

Checklist previa (5 minutos antes)

- **Pre-calentar** (el plan free duerme los servicios a los 15 min; despiertan en ~1 min):

```
for s in legacy clientes proveedores productos ventas pagos despacho tickets feedback notificaciones administrador; do
  curl -s -o /dev/null -w "paris-$$: ${http_code}\n" --max-time 120 https://paris-$.onrender.com/v3/api-docs &
done; wait      # repetir hasta que los 11 den 200
```

- Datos ya cargados en Neon: admin, 2 vendedores APROBADOS con un taladro cada uno, oferta 20% en el de Cóndor, 5.000 clientes legacy.
- Elegir la María de hoy: un `clienteN@paris.cl` / `passN` **aún no migrado** (llevar anotado el N).

Credenciales de la demo

Actor	Login	Credenciales	Rol JWT
María (cliente histórico)	POST paris-clientes /api/v1/clientes/login	clienteN@paris.cl / passN	CLIENTE
Ferretería Cóndor (vendedor)	POST paris-proveedores /api/v1/proveedores/login	condor@ferreteria.cl / condor123	PROVEEDOR
Administrador	POST paris-administrador /api/v1/administradores/login	admin / admin123	ADMINISTRADOR

Guardar cada token y usarlo como `Authorization: Bearer <token>`. En los ejemplos: `$TM` (María), `$TC` (Cóndor), `$TA` (admin).

Acto 1 — «Ingresa con su correo habitual» (validación contra la API legacy)

```
POST https://paris-clientes.onrender.com/api/v1/clientes/login
{"email": "cliente31@paris.cl", "password": "pass31"}
```

Qué destacar: María existe solo en el legacy. La respuesta trae `"tipo": "MIGRADO"` y `"legacyId": 31` — `clientes` validó las credenciales contra `legacy` vía WebClient y la migró **sin re-registro** (password re-almacenada con BCrypt). El `token` devuelto es su JWT rol CLIENTE. Guardar también su `id` (→ `$MARIA_ID`).

Acto 2 — «Busca un taladro, elige entre ofertas de distintos vendedores»

```
GET https://paris-productos.onrender.com/api/v1/productos?categoria=Herramientas (público)
GET https://paris-feedback.onrender.com/api/v1/resenas/producto/1/promedio (público)
```

Qué destacar: dos taladros de **vendedores distintos**. El de Cóndor (id 1) vale \$50.000 de lista pero su `precioVigente` es **\$40.000** por la oferta del 20% activa; el de Toolmax (id 2) vale \$45.000. Con la reputación (promedio de reseñas con compra verificada), María elige el de Cóndor.

Acto 3 — «Paga» (validaciones cruzadas + orquestación)

```
POST https://paris-ventas.onrender.com/api/v1/ventas (Bearer $TM)
{"clienteId": $MARIA_ID, "detalles": [{"productoId": 1, "cantidad": 1}]}
```

Qué destacar: `ventas` validó el cliente (→ `clientes`) y el producto/stock (→ `productos`); tomó el **precio vigente** como snapshot y calculó la **comisión del 10% en el servidor**: `montoTotal: 40000`, `comisionTotal: 4000`. Guardar el id de la venta (→ `$VENTA_ID`).

```
PATCH https://paris-ventas.onrender.com/api/v1/ventas/$VENTA_ID/pagar (Bearer $TM)
{"medioPago": "TARJETA", "direccionEntrega": "Av. Providencia 123, Santiago"}
```

Qué destacar: un solo PATCH **orquestó 4 servicios**: productos (stock 6 → 5), pagos (pago + comprobante con folio `F-000000NN`), despacho (seguimiento `ENV-000000NN`) y notificaciones (aviso a María). Evidencias:

```
GET paris-productos /api/v1/productos/1 → stock descontado
GET paris-pagos /api/v1/pagos?ventaId=$VENTA_ID (Bearer $TM) → pago; guardar id (→ $PAGO_ID)
GET paris-pagos /api/v1/pagos/$PAGO_ID/comprobante → folio único
```

Acto 4 — «Sigue el estado del despacho desde su perfil»

```
GET https://paris-despacho.onrender.com/api/v1/envios?clienteId=$MARIA_ID (Bearer $TM)
```

El **vendedor** (token de Cóndor) despacha — acción exclusiva del rol `PROVEEDOR`:

```
PATCH paris-despacho /api/v1/envios/$ENVIO_ID/estado (Bearer $TC)
{"estado": "ENVIADO", "comentario": "Retirado por courier"}
PATCH paris-despacho /api/v1/envios/$ENVIO_ID/estado (Bearer $TC)
{"estado": "ENTREGADO", "comentario": "Recibido conforme"}
```

```
GET paris-despacho /api/v1/envios/$ENVIO_ID/historial (Bearer $TM)
GET paris-notificaciones /api/v1/notificaciones?destinatarioTipo=CLIENTE&destinatarioId=$MARIA_ID (Bearer $TM)
```

Qué destacar: trazabilidad completa `PENDIENTE` → `ENVIADO` → `ENTREGADO` con fecha y comentario de cada cambio; al pasar a `ENVIADO`, `despacho` disparó el aviso de `DESPACHO` vía `WebClient`. La bandeja de María muestra `VENTA_CONFIRMADA` y `DESPACHO`.

Acto 5 — «El producto llega defectuoso: abre un reclamo»

```
POST https://paris-tickets.onrender.com/api/v1/tickets (Bearer $TM)
{"ventaId": $VENTA_ID, "categoria": "PRODUCTO_DEFECTUOSO",
 "asunto": "Taladro no enciende", "descripcion": "El gatillo llegó trabado, no gira"}
```

Qué destacar: `tickets` validó contra `ventas` que la compra es **real** y tomó de ella el cliente y el proveedor (María no los envió). Queda `ABIERTO`. Guardar el id (→ `$TICKET_ID`).

Acto 6 — «El Administrador media la disputa y autoriza el reembolso»

```
POST paris-tickets /api/v1/tickets/$TICKET_ID/mensajes (Bearer $TA)
{"autorRol": "ADMINISTRADOR", "mensaje": "Estamos revisando el caso con el vendedor"}
POST paris-tickets /api/v1/tickets/$TICKET_ID/mensajes (Bearer $TC)
{"autorRol": "PROVEEDOR", "mensaje": "Aceptamos la devolución, lote con falla"}
```

Qué destacar: la primera intervención del admin deja el ticket `EN_MEDIACION` — el hilo entre cliente, vendedor y administrador ES la mediación.

```
PATCH paris-tickets /api/v1/tickets/$TICKET_ID/resolver (Bearer $TA)
{"estado": "RESUELTO", "resolucion": "Falla de fábrica confirmada; se reembolsa el total",
 "reembolsoAutorizado": true}
```

```
GET paris-pagos /api/v1/pagos/$PAGO_ID/reembolsos (Bearer $TA) → PROCESADO, con ticketId
GET paris-pagos /api/v1/pagos/$PAGO_ID (Bearer $TA) → estado REEMBOLSADO
GET paris-notificaciones /api/v1/notificaciones?destinatarioTipo=CLIENTE&destinatarioId=$MARIA_ID&tipo=RESOLUCION_RECLAMO
```

Qué destacar: solo el rol ADMINISTRADOR puede resolver; `tickets` invocó a `pagos` (reembolso por el monto completo, con referencia al ticket, pago `REEMBOLSADO`) y avisó a María. **La historia quedó cerrada de extremo a extremo.**

Si algo se cae en vivo (plan B)

- **Timeout / 503 al primer intento:** un servicio se durmió — reintentar la misma petición (despierta en ~1 min) o repetir el pre-calentado.
- **409 en el login de María:** ese clienteN ya fue migrado en un ensayo — usar el siguiente (N+1).
- **Swagger como respaldo visual:** <https://paris-<servicio>.onrender.com/swagger-ui/index.html> — cada endpoint documenta validaciones, roles y códigos (@Operation/@ApiResponse/@RequestBody/@ExampleObject).
- **RBAC en vivo (si preguntan por seguridad):** repetir cualquier petición sin token → 401/403; o intentar publicar un producto con el token de María → 403.